Genesis Intelligence Architecture: A Lightweight Knowledge-First Approach to Experience-Driven Artificial Intelligence

Learn by Experience, Not by Retraining.

[Author Name Placeholder]
[Institution / Affiliation Placeholder]
[Email Placeholder]

ABSTRACT

Traditional artificial intelligence systems rely heavily on pretrained neural-network parameters and computationally expensive training cycles. Large language models (LLMs), despite their impressive capabilities, require substantial GPU resources for training, suffer from knowledge cutoffs, and cannot easily acquire new knowledge without retraining or external retrieval. This paper proposes the **Genesis Intelligence Architecture (GIA)**, a novel experimental architecture that shifts the primary mechanism of growth from increasing model weights to persistent memory, structured knowledge graphs, experience accumulation, verification, reasoning, skill acquisition, and controlled autonomous learning. GIA is designed to operate on ordinary CPU hardware and to remain useful both offline and online. When local knowledge is insufficient, the system can perform web research, verify retrieved information, transform it into structured knowledge, store it locally, and reuse it in future tasks. Importantly, this does not constitute conventional neural-network training; rather, it is a knowledge-first, experience-driven approach to building progressively more capable AI systems without continuously retraining a large neural model. This paper presents GIA as a research proposal and prototype architecture, clearly distinguishes existing technologies from the proposed contribution, and outlines an experimental design for future empirical validation. The architecture is experimental and has not yet demonstrated human-level intelligence or superiority over existing systems.

Keywords: *knowledge-first AI, experience-driven learning, persistent memory, knowledge graphs, continual learning, neuro-symbolic AI, autonomous agents, offline AI, memory-augmented systems, skill acquisition*

I. Introduction

Artificial intelligence has undergone remarkable advances over the past decade, driven primarily by deep learning and, more recently, by transformer-based large language models. The transformer architecture, introduced by Vaswani et al. [1], revolutionised natural language processing by replacing recurrent layers with self-attention mechanisms, enabling greater parallelism and improved translation quality. Building on this foundation, models such as GPT-3, with 175 billion parameters [2], demonstrated that scaling model size yields substantial improvements in task-agnostic, few-shot performance across a wide range of natural language tasks.

Despite these achievements, current AI systems face significant limitations. Training a large language model requires enormous computational resources, typically involving arrays of GPUs running for days or weeks. Once trained, the model parameters are static: the knowledge encoded in the weights becomes increasingly stale as the world changes. This gives rise to the *knowledge cutoff* problem, where the model has no awareness of events, facts, or developments that occurred after its training data was collected. Techniques such as retrieval-augmented generation (RAG) [3] partially address this by fetching relevant documents at inference time, but the model itself does not retain what it retrieves from one interaction to the next.

Agent systems, which combine LLMs with external tools, memory, and multi-step reasoning [4], represent a step toward more dynamic AI. Continual learning and lifelong learning research [5], [6] seek to address the broader challenge of enabling models to accumulate knowledge over time without catastrophic forgetting. Memory-augmented neural networks [7], [8] explore architectures that couple neural processing with external memory resources. Neuro-symbolic AI [9], [10] investigates the integration of symbolic reasoning with neural learning. These research directions collectively motivate a question: is there an alternative path to progressively more capable AI that does not depend solely on increasing neural model size?

This paper introduces the **Genesis Intelligence Architecture (GIA)**, a proposed experimental framework that explores this question. GIA is designed around the principle that intelligence can be partially represented as organised knowledge, structured memory, relationships, procedures, and accumulated experience, rather than being entirely encoded in neural-network weights. The architecture incorporates persistent memory, a knowledge graph, verification mechanisms, reasoning, skill acquisition, feedback loops, and controlled autonomous learning. It is designed to run on commodity CPU hardware and to function both offline and online.

Research Question: Can an AI system become progressively more capable through structured experience, memory, verification, and skill acquisition without continuously retraining a large neural model?

We emphasise that GIA is presented as a research proposal and prototype architecture. It does not claim to have achieved human-level intelligence, nor does it claim to replace existing large language models. The goal is to establish a scientifically credible foundation that can be experimentally tested and potentially developed into a new AI research direction.

II. Problem Statement

Current AI systems, despite their successes, face a constellation of interconnected problems that motivate the search for alternative architectures:

- **Expensive model training:** Training state-of-the-art LLMs requires large clusters of GPUs and substantial energy consumption, making AI development accessible primarily to well-resourced organisations [1], [2].
- **GPU requirements:** Both training and, for the largest models, inference depend on specialised GPU hardware that is costly and subject to supply-chain constraints.
- **Static model parameters:** Once trained, a model weights are frozen. New knowledge cannot be incorporated without retraining or fine-tuning, which is expensive and risks degrading existing capabilities.
- **Knowledge updates:** The world changes continuously, but a trained model has no mechanism to update its internal knowledge without retraining, leading to staleness.
- **Repeated computation:** When a user asks a similar question multiple times, the model recomputes the answer from scratch each time, with no efficiency gain from prior interactions.
- **Lack of persistent personal memory:** Most AI systems do not maintain a memory of past interactions with a specific user, preventing personalisation and contextual continuity.
- **Difficulty maintaining long-term knowledge:** Continual learning research [5] has identified catastrophic forgetting as a fundamental challenge; neural networks tend to lose previously learned knowledge when learning new tasks.
- **Hallucination:** LLMs can generate fluent but factually incorrect outputs, a problem rooted in the statistical nature of language generation [2].
- **Knowledge conflicts:** When retrieved information contradicts a model internal knowledge, there is no robust mechanism for resolving the conflict or determining which source is more reliable.
- **Privacy concerns:** Cloud-based AI systems typically transmit user data to remote servers, raising privacy issues. Users have limited control over what is stored, for how long, and how it is used.

These problems are not independent; they interact. The static nature of model parameters drives the knowledge-cutoff problem, which in turn motivates retrieval mechanisms that introduce their own challenges around verification and conflict resolution. The computational cost of training creates a barrier to frequent updates, exacerbating staleness. GIA proposes to address these problems not by making models larger, but by rethinking where intelligence resides.

III. Research Hypothesis

We propose the following hypothesis:

> *If intelligence is partially represented as organised knowledge, memory, relationships, procedures, and accumulated experience, then some forms of AI capability may improve through structured memory growth and reasoning without retraining the primary neural parameters.*

This hypothesis is grounded in several observations. First, human intelligence is not purely parametric; humans accumulate knowledge, skills, and experience throughout their lives without retraining their neural architecture from scratch. Second, knowledge graphs have demonstrated that structured, relational representations of knowledge can support reasoning, question answering, and knowledge discovery [11]. Third, memory-augmented systems have shown that coupling external memory with processing can enable capabilities beyond what is

stored in model weights alone [7], [8].

We explicitly state that this is a hypothesis that requires experimental validation. The architecture proposed in this paper is designed to enable such experimentation, not to assert that the hypothesis has been confirmed. The extent to which structured knowledge growth can substitute for, or complement, neural parameter updates remains an open empirical question.

IV. Existing Approaches

A. Artificial Neural Networks

Artificial neural networks (ANNs), inspired by the structure of biological neurons, learn by adjusting weighted connections between layers of nodes through backpropagation. Deep neural networks achieved breakthroughs in image recognition, speech processing, and game playing. Their strength lies in learning complex patterns from large datasets, but their knowledge is encoded in distributed weight values that are difficult to inspect, update selectively, or interpret.

B. Large Language Models

Large language models such as GPT-3 [2] are autoregressive neural networks trained on massive text corpora. GPT-3, with 175 billion parameters, demonstrated strong few-shot performance across diverse NLP tasks including translation, question-answering, and arithmetic. However, LLMs store knowledge implicitly in their weights, cannot be easily updated, and can hallucinate factually incorrect information. Their training cost is substantial, and their knowledge has a fixed cutoff date.

C. Retrieval-Augmented Generation

RAG, introduced by Lewis et al. [3], combines pre-trained parametric memory (a seq2seq model) with non-parametric memory (a dense vector index of documents, such as Wikipedia, accessed via a neural retriever). RAG models generate more specific, diverse, and factual language than parametric-only baselines and set state-of-the-art results on open-domain QA. However, RAG does not persist what it retrieves; each query begins fresh, and the retrieved documents are not transformed into structured, reusable knowledge.

D. Knowledge Graphs

Knowledge graphs represent information as entities and relationships in a graph structure. Hogan et al. [11] provide a comprehensive survey, covering graph-based data models, query languages, validation, and deductive and inductive knowledge extraction techniques. Knowledge graphs enable structured representation, querying, and reasoning over facts, and have been adopted widely in industry. They complement neural models by providing explicit, inspectable, and updatable knowledge.

E. Expert Systems and Symbolic AI

Expert systems, prominent in the 1970s-80s, encode domain knowledge as explicit rules and inference engines. Symbolic AI represents knowledge using formal logic, enabling precise reasoning and explainability. However, symbolic systems struggle with uncertainty, require manual knowledge engineering, and lack the learning capabilities of neural approaches. Their strengths (explicitness, inspectability) and weaknesses (brittleness, manual effort) complement those of neural networks.

F. Agentic AI

ReAct, proposed by Yao et al. [4], synergises reasoning and acting in language models by generating interleaved reasoning traces and actions. This allows LLMs to interact with external environments (e.g., Wikipedia APIs) and overcome hallucination issues present in pure chain-of-thought reasoning. Agent systems represent a significant step toward dynamic, tool-using AI, but they typically do not persist learned knowledge across sessions.

G. Continual and Lifelong Learning

Continual learning addresses the challenge of learning from non-stationary data distributions over time. De Lange et al. [5] survey continual learning methods, noting that catastrophic forgetting, where learning new tasks degrades performance on old ones, is a central challenge. Wang et al. [6] provide a comprehensive survey covering theory, methods, and applications, identifying the stability-plasticity trade-off and generalisation as key objectives. These fields seek to make neural models more adaptable over time, but the fundamental approach remains weight-based learning.

H. Memory-Augmented Neural Networks

Memory Networks, introduced by Weston et al. [7], couple neural networks with large addressable memory that can be read and written to, enabling reasoning over long sequences. Neural Turing Machines (NTMs), proposed by Graves et al. [8], extend neural networks with external memory resources accessed via attention, analogous to a Turing machine. These architectures demonstrate that explicit memory can complement neural processing, but the memory is typically differentiable and trained end-to-end, rather than being a persistent, structured knowledge store.

I. Neuro-Symbolic AI

Neuro-symbolic AI integrates symbolic reasoning with neural learning. Recent surveys [9], [10] describe how this integration can yield systems with improved accuracy, reduced data requirements through background knowledge integration, and greater explainability. Four key aspects are discussed: representation, learning, reasoning, and decision-making. Neuro-symbolic approaches span composite frameworks (keeping symbolic and neural components separate) and monolithic frameworks (integrating reasoning into neural architectures). GIA draws inspiration from neuro-symbolic principles but differs in its emphasis on persistent, experience-driven knowledge growth rather than end-to-end training.

J. Autonomous Agents

Autonomous agent systems combine language models with planning, tool use, and environmental interaction. These systems can perform multi-step reasoning, access external APIs, and complete complex tasks. However, they typically rely on a large underlying LLM for each step, do not maintain persistent structured knowledge across sessions, and inherit the computational requirements of their base models.

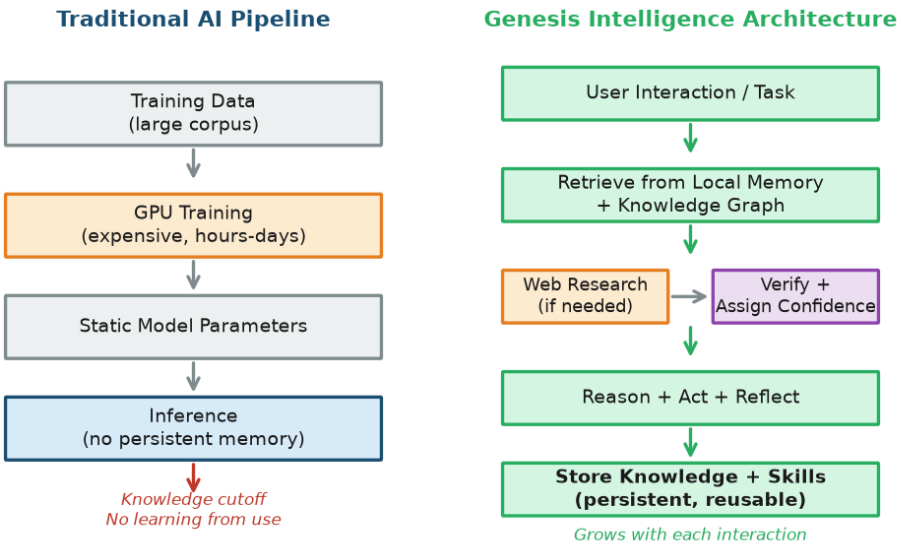

Fig. 1. Traditional AI pipeline (left) versus the Genesis Intelligence Architecture (right). Traditional AI trains static parameters and performs inference without persistent memory. GIA retrieves from local memory, researches when needed, verifies, and stores knowledge for future reuse.

V. Proposed Genesis Intelligence Architecture

The Genesis Intelligence Architecture (GIA) proposes a knowledge-first, experience-driven approach to AI. Rather than treating model weights as the sole substrate of intelligence, GIA distributes intelligence across persistent memory, a structured knowledge graph, reasoning mechanisms, acquired skills, and accumulated experience. The architecture is designed so that the system becomes incrementally more capable with each interaction, without retraining any neural model.

At the heart of GIA is a twelve-stage learning loop that governs how the system processes each interaction, acquires new knowledge, and improves over time. The loop is designed to be both reactive (responding to user requests) and proactive (filling knowledge gaps during idle periods).

A. The Core Learning Loop

OBSERVE: The system receives input from the user, environment, or its own curiosity engine. This includes natural language queries, task requests, feedback signals, or detected knowledge gaps.

UNDERSTAND: The Intent Engine parses the input to determine what the user is asking for, disambiguating language and identifying the type of task (question, creation, analysis, etc.).

RETRIEVE: The system queries its local memory and knowledge graph for relevant existing knowledge, skills, and past experiences. If sufficient knowledge is available, the system proceeds directly to reasoning.

RESEARCH: If local knowledge is insufficient, the Web Research Engine searches external sources, retrieves documents, and extracts candidate claims for further processing.

VERIFY: The Verification Engine cross-compares retrieved claims, assesses source reliability, detects contradictions, and assigns confidence scores before any knowledge is accepted.

LEARN: Verified information is transformed into structured knowledge units: concepts, relationships, evidence, and confidence scores. The Learning Engine identifies what is new, what updates existing knowledge, and what conflicts with it.

REMEMBER: New and updated knowledge is stored in the appropriate memory systems: episodic memory for events, semantic memory for facts, procedural memory for methods, and skill memory for reusable capabilities.

CONNECT: The system creates and strengthens relationships in the knowledge graph, linking new concepts to existing ones, identifying patterns, and building a richer relational structure.

REASON: The Reasoning Engine applies logical inference, analogy, and pattern matching over the knowledge graph to derive answers, plans, or solutions. This may involve multi-step reasoning chains.

ACT: The system executes the appropriate response: answering a question, generating code, performing a task via the Tool Engine, or creating a deliverable.

REFLECT: The Reflection Engine evaluates the outcome: Was the response correct? Was the user satisfied? Did the task succeed? Feedback is captured for future improvement.

IMPROVE: Based on reflection and feedback, the system updates its skills, adjusts confidence scores, refines its knowledge graph, and identifies areas for future learning. The system is now incrementally more capable than before.

This loop is not strictly linear; stages may iterate (e.g., RESEARCH and VERIFY may cycle multiple times) or be skipped (e.g., RETRIEVE may provide sufficient knowledge, bypassing RESEARCH). The loop cyclical nature is what enables progressive improvement.

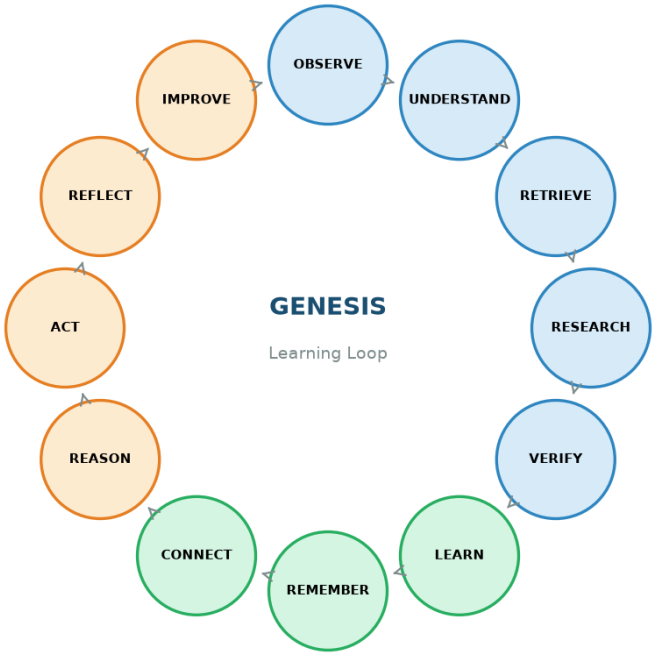

Fig. 3. The GIA learning loop. Twelve stages from observation to improvement form a continuous cycle. Each iteration potentially increases the system knowledge and skill base.

VI. Architectural Components

GIA comprises fifteen modular components, organised into five functional layers: interaction, knowledge, control, learning, and growth. Each component has defined responsibilities and interacts with others through the central orchestrator.

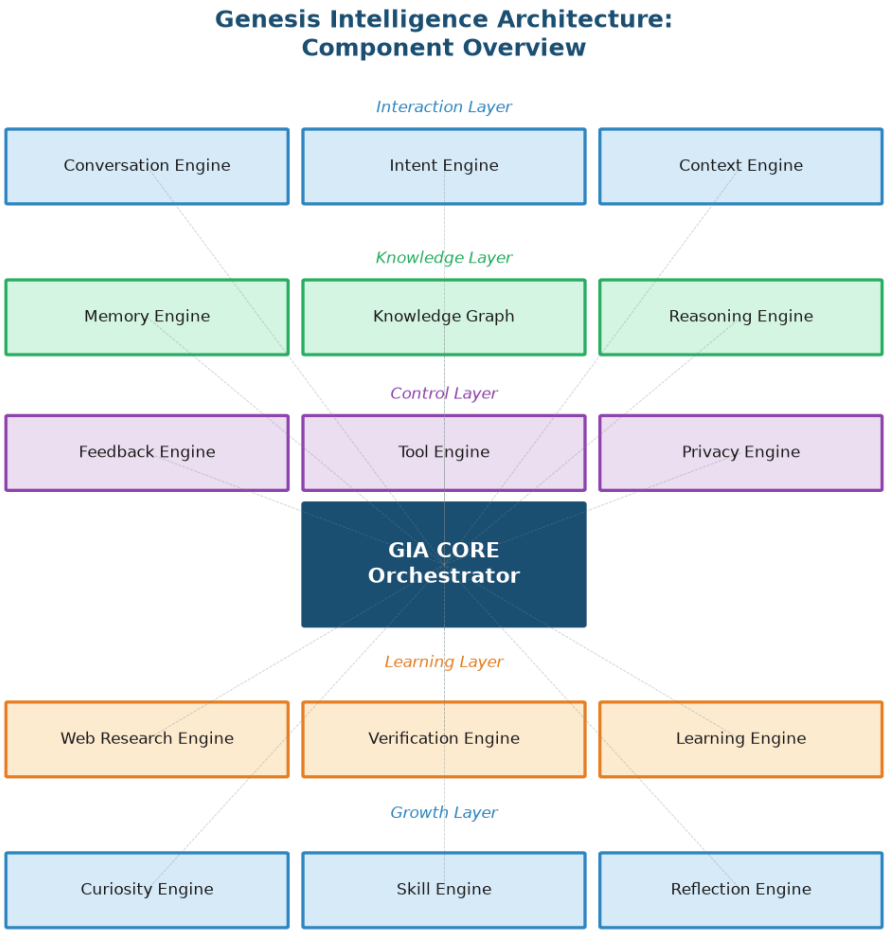

Fig. 2. Genesis Intelligence Architecture: fifteen components organised into five layers. The central orchestrator coordinates all components.

1. Conversation Engine: Manages the dialogue interface between the user and the system. It handles input parsing, output formatting, multi-turn conversation tracking, and maintains the current interaction context.

2. Intent Engine: Analyses user input to determine the underlying intent: question answering, code generation, analysis, learning, or task execution. It disambiguates language, identifies entities, and classifies the request type.

3. Context Engine: Maintains the active context window: what the user is working on, what was discussed recently, what files or data are relevant. It provides situational awareness that informs retrieval and reasoning.

4. Memory Engine: Manages all memory operations across short-term, long-term, episodic, semantic, procedural, and skill memory. It handles storage, retrieval, consolidation, and forgetting of information across memory types.

5. Knowledge Graph: A structured graph of concepts, relationships, evidence, and confidence scores. It is the system primary structured knowledge representation, enabling relational reasoning, pattern discovery, and knowledge querying.

6. Reasoning Engine: Performs logical inference, analogy, deduction, and pattern matching over the knowledge graph and memory. It can chain reasoning steps, identify relevant prior experiences, and derive conclusions from available evidence.

7. Web Research Engine: Activated when local knowledge is insufficient. It constructs search queries, retrieves web documents, and extracts candidate claims. It interfaces with external search APIs and document retrieval systems.

8. Verification Engine: Evaluates retrieved claims by cross-comparing multiple sources, assessing source reliability, detecting contradictions, and assigning confidence scores. No external knowledge enters the knowledge graph without verification.

9. Learning Engine: Transforms verified information into structured knowledge units and integrates them into the knowledge graph and memory systems. It identifies what is new, what updates existing knowledge, and what conflicts with it.

10. Curiosity Engine: During idle periods, it identifies knowledge gaps, prioritises useful topics for research, and initiates autonomous learning cycles. It operates under strict resource constraints and user permissions.

11. Skill Engine: Creates, stores, and retrieves reusable skills, procedures, and methods. When a task is successfully completed, the Skill Engine abstracts the solution into a reusable skill.

12. Reflection Engine: Evaluates the outcome of each interaction: correctness, user satisfaction, task success, and efficiency. It identifies what went well, what failed, and what could be improved, feeding this analysis back into the system.

13. Feedback Engine: Collects explicit and implicit feedback from users (corrections, ratings, behavioural signals) and integrates it into the system knowledge and confidence assessments.

14. Tool Engine: Manages the execution of external tools and APIs: code execution, file operations, web requests, and third-party integrations. It operates under the Privacy Engine constraints and includes safety checks before any tool execution.

15. Privacy Engine: Enforces local-first storage policies, controls what data leaves the system, manages user consent for autonomous learning and web research, and provides mechanisms for data inspection, modification, and deletion.

VII. Knowledge Representation

A central design decision in GIA is that the system does not store only raw text. While text is the medium of communication, it is a poor substrate for reasoning, verification, and reuse. GIA instead represents knowledge as structured units that capture not only what is known, but how it is known, how confident the system is, and where the knowledge came from.

A. Knowledge Unit Structure

Field	Description	Example
Concept	The entity or idea being represented	Python
Relationship	How this concept relates to others	is-a Programming Language
Evidence	Supporting source or reasoning chain	python.org; Python docs
Confidence	Numeric score [0,1] based on verification	0.92
Timestamp	When the knowledge was acquired/updated	2026-09-05T14:30
Source	Origin of the knowledge	web:python.org / user / derived
Status	Active, superseded, or contradicted	active

Table I. Knowledge unit fields in GIA structured representation.

This structure enables several capabilities that raw text storage cannot provide. **Confidence scores** allow the system to reason about uncertainty and prioritise high-confidence knowledge. **Source tracking** provides provenance, enabling verification and source-reputation assessment. **Timestamps** enable knowledge aging, the gradual decay of confidence for time-sensitive facts. **Status fields** allow the system to track how knowledge evolves, maintaining a history of what was superseded and why.

B. Example: Python Knowledge Graph

The following example illustrates how knowledge about the Python programming language and GUI development is represented as a connected subgraph:

```
Python
  |
  +-- is-a --> Programming Language
  +-- supports --> GUI Development
  +-- has-library --> Tkinter
  +-- property --> Cross-Platform

Tkinter
  |
  +-- enables --> GUI Development
  +-- used-in --> Calculator App
  +-- stored-as --> Skill: GUI_Calc
```

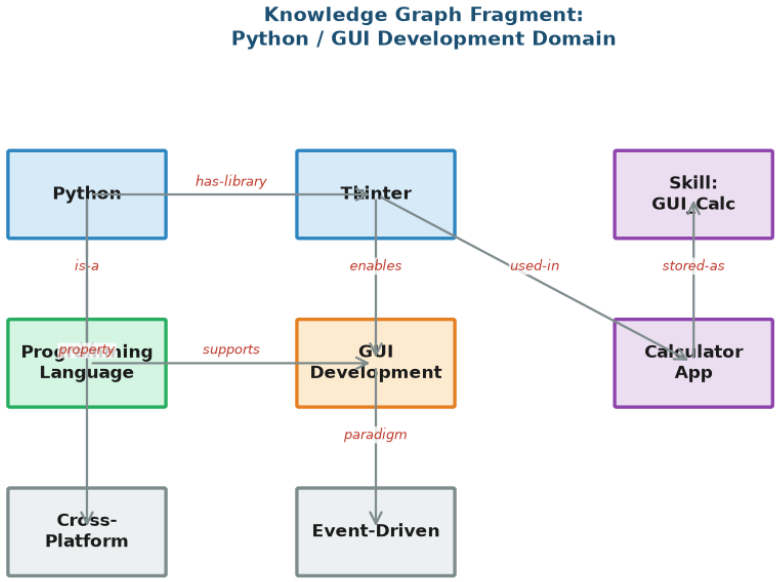

Fig. 4. A fragment of GIA knowledge graph showing concepts (nodes) and typed relationships (edges) in the Python/GUI development domain.

In this representation, asking whether Python can create a GUI is answered not by retrieving a text passage, but by traversing the graph: Python supports GUI Development, Tkinter enables GUI Development, and Tkinter is a library of Python. The reasoning is explicit, inspectable, and reusable.

VIII. Memory Architecture

GIA incorporates five types of memory, inspired by cognitive science models of human memory. Each type serves a distinct function and contributes differently to the system overall intelligence.

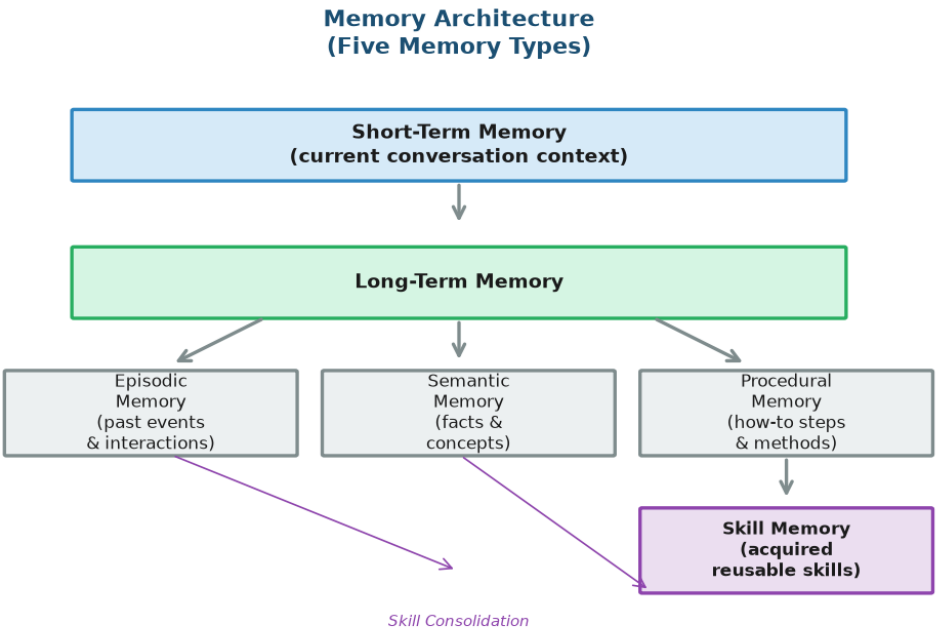

Fig. 5. GIA five-type memory architecture. Short-term and long-term memory are organised hierarchically, with long-term memory comprising episodic, semantic, procedural, and skill memory.

Short-term memory: Maintains the current conversation context: recent messages, active entities, and immediate task state. It is volatile, limited in capacity, and continuously updated. Short-term memory enables coherent multi-turn dialogue and contextual awareness within a session.

Long-term memory: The persistent store that survives across sessions. It is the umbrella for the following three types, plus skill memory. Long-term memory is what enables the system to become more capable over time.

Episodic memory: Records specific past events and interactions: what was asked, what was answered, what tools were used, and what the outcome was. Episodic memory enables the system to recall similar past situations and apply lessons learned.

Semantic memory: Stores facts, concepts, and their relationships, independent of when or how they were learned. This is the knowledge graph primary substrate. Semantic memory enables general knowledge retrieval and relational reasoning.

Procedural memory: Contains methods, algorithms, step-by-step procedures, and how-to knowledge. When the system learns how to perform a task, the procedure is stored here.

Skill memory: A specialised form of procedural memory that stores abstracted, reusable skills created from successful task completions. Skills are the consolidated output of the learning loop, representing generalised solutions that can be applied to new but similar problems.

The interplay between these memory types is crucial. Episodic memory provides the raw material from which procedural memory and skill memory are abstracted. Semantic memory provides the conceptual framework that connects episodic experiences. Short-term memory provides the working context in which all other memory types are accessed and updated. Together, they form a multi-layered memory system that, we hypothesise, can support progressive capability growth.

IX. Experience-Driven Learning

Experience-driven learning is the mechanism by which GIA converts interactions into reusable capabilities. The process can be illustrated through a concrete example.

A. The Learning Process

When a user asks the system to create a Python calculator, the following sequence occurs:

1. The user asks: "Create a Python calculator."

2. The system retrieves relevant knowledge from its knowledge graph (Python syntax, arithmetic operations, input/output handling). If gaps exist, it researches and verifies missing information.

3. The system generates a calculator implementation, possibly executing test cases via the Tool Engine.

4. The result is recorded in episodic memory: the query, the approach taken, the code generated, and whether it succeeded.

5. If successful, the Reflection Engine identifies the effective strategy: which parsing approach worked, how arithmetic was handled, what structure was used.

6. The Learning Engine extracts concepts (e.g., arithmetic parsing, operator precedence, input validation) and creates relationships in the knowledge graph.

7. The Skill Engine abstracts the solution into a reusable skill: "python_calculator_creation", storing the method, relevant concepts, and success conditions.

8. On a future request to create a similar tool, the system retrieves and applies the skill, potentially reducing research time and improving output quality.

B. Example: Progressive Calculator Tasks

Consider a sequence of related tasks that demonstrate how accumulated experience reduces future effort:

Day	Task	New Knowledge/Skill Acquired	Reused From
1	Create Python calculator	Arithmetic parsing, operator precedence, CLI I/O	-
2	Create GUI calculator	Tkinter layout, button event binding	Day 1: arithmetic logic
3	Add history feature	State management, history stack	Days 1-2: calculator + GUI structure
7	Create scientific calculator	Math functions, expression evaluation	Days 1-3: full calculator + history skill

Table II. Progressive calculator tasks showing knowledge and skill accumulation over time.

By Day 7, the system has accumulated a rich set of concepts and skills. Creating a scientific calculator requires primarily *new* knowledge (math functions, expression evaluation), while the foundational structure (calculator logic, GUI, history) is retrieved from existing skills. This is the hypothesised mechanism by which GIA becomes progressively more capable: each task builds on accumulated experience, reducing the marginal cost of solving related problems.

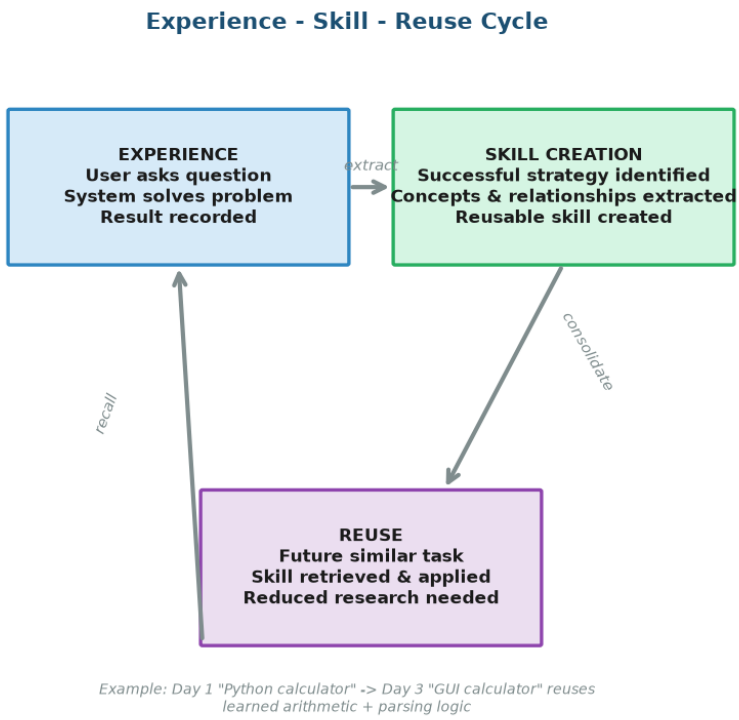

Fig. 8. The experience-skill-reuse cycle. Experience from solving a problem is abstracted into a reusable skill, which is then recalled for similar future tasks.

X. Internet Learning

GIA is designed with an **offline-first** philosophy: local knowledge is always consulted first. Only when the system existing knowledge is insufficient to address a query does it seek external information. This design minimises unnecessary network usage, reduces latency for repeated queries, and respects user privacy.

A. The Internet Learning Pipeline

When a knowledge gap is detected, the following pipeline is activated:

- **Search**: The Web Research Engine constructs multiple search queries targeting the knowledge gap, using different phrasings to maximise recall.
- **Retrieve sources**: Candidate documents are retrieved from web search results. The system prioritises sources with established reliability (academic publications, official documentation, reputable references).
- **Extract claims**: The system extracts discrete factual claims from retrieved documents, identifying assertions, definitions, relationships, and data points.
- **Compare**: Extracted claims are cross-compared to identify agreements, contradictions, and gaps. Multiple independent sources supporting the same claim increase confidence.
- **Verify**: The Verification Engine assesses each claim against source reliability, internal consistency, and existing knowledge. Claims that contradict high-confidence existing knowledge are flagged for review.
- **Assign confidence**: A confidence score is assigned to each verified claim, based on source reputation, corroboration level, and consistency with existing knowledge.
- **Store**: Verified knowledge is integrated into the knowledge graph and appropriate memory systems, with full provenance (source, timestamp, confidence).

B. Source Reliability and Contradiction Detection

Not all sources are equally reliable. GIA maintains a source reputation model that assigns reliability scores to different source types. Academic publications, official documentation, and established encyclopedic sources receive higher base reliability than user-generated content, blogs, or unvetted web pages. Reputation scores are adjusted over time based on verification outcomes: sources whose claims are consistently corroborated gain reputation; sources whose claims are frequently contradicted lose it.

Contradiction detection is critical. When two sources provide conflicting information, the system does not simply accept the more recent claim. Instead, it records the contradiction, compares source reliability scores, and assigns a lower confidence to the conflicting knowledge. If the contradiction involves high-confidence existing knowledge, the system may flag it for user review rather than automatically overwriting. This conservative approach is designed to prevent knowledge poisoning and maintain trust in the knowledge graph.

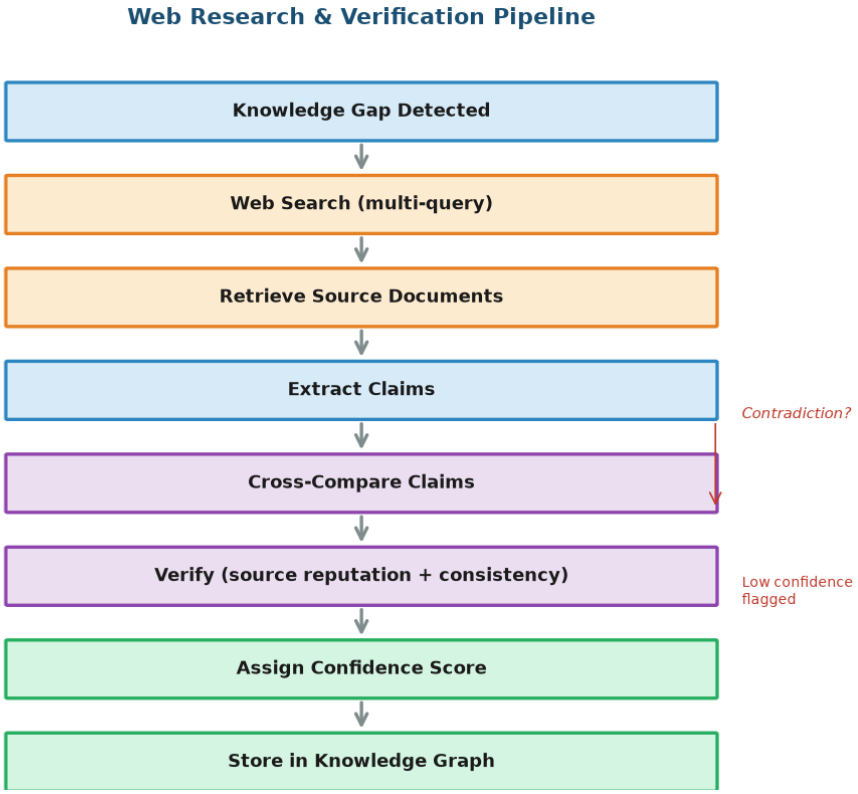

Fig. 6. The web research and verification pipeline. External claims are searched, retrieved, extracted, cross-compared, verified, confidence-scored, and stored only after passing verification.

XI. Autonomous Learning

The Curiosity Engine enables GIA to learn proactively during idle periods, rather than only reactively when a user asks a question. This is analogous to how humans consolidate and expand knowledge during quiet moments. When the system detects that it is idle (no active user interaction), the Curiosity Engine identifies knowledge gaps and initiates controlled research to fill them.

A. Curiosity Engine Operation

The Curiosity Engine operates through the following steps:

- **Identify knowledge gaps**: The system analyses its knowledge graph to identify areas where coverage is thin, where confidence is low, or where gaps were encountered during past interactions.
- **Prioritise useful topics**: Gaps are ranked by utility, based on factors such as frequency of past queries, relevance to user interests, and connectivity to existing knowledge.
- **Research**: The Web Research Engine is invoked to fill the highest-priority gaps, following the same pipeline described in Section X.
- **Verify**: Retrieved information passes through the Verification Engine before integration.
- **Update knowledge graph**: Verified knowledge is integrated, relationships are created, and confidence scores are assigned.

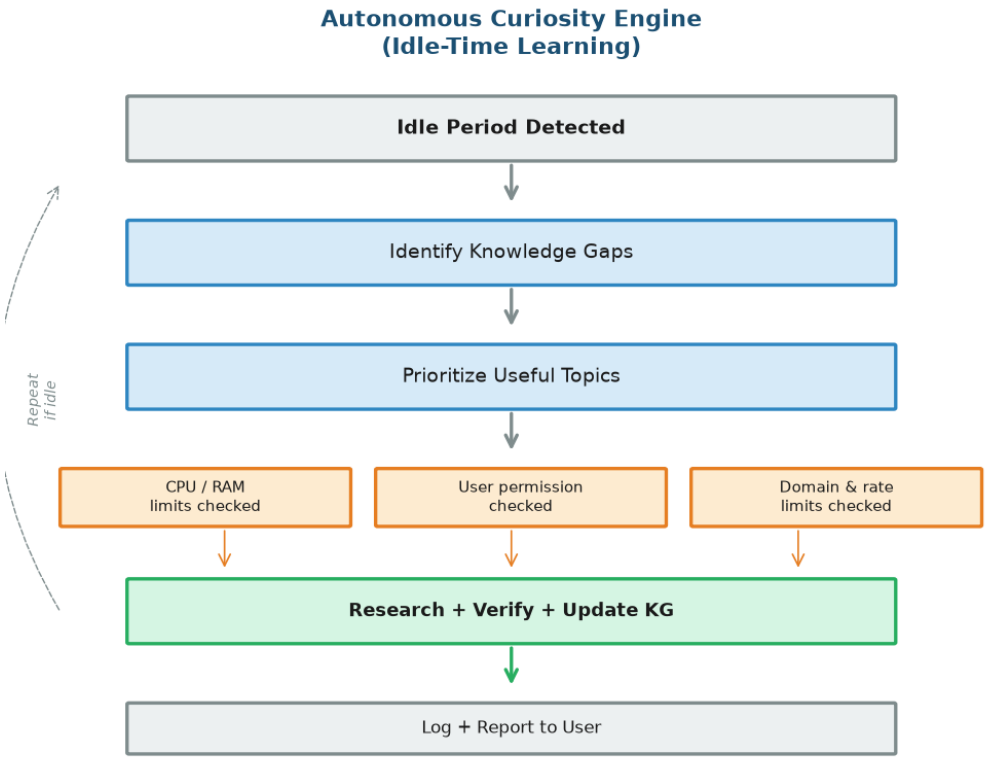

Fig. 7. The autonomous Curiosity Engine. During idle periods, the system identifies and fills knowledge gaps under strict resource and permission controls.

B. Limitations and Safety Controls

Autonomous learning introduces risks that must be carefully managed. GIA incorporates strict controls to ensure that autonomous learning is safe, bounded, and user-controlled:

- **CPU limits**: Autonomous learning is throttled to a configurable maximum CPU utilisation, ensuring it does not interfere with system responsiveness.
- **RAM limits**: Memory usage during autonomous learning is bounded to prevent resource exhaustion.
- **Network limits**: The frequency and volume of web requests during autonomous research are rate-limited to avoid excessive bandwidth usage.
- **Storage limits**: Knowledge graph growth is monitored, with configurable maximum storage size. The system can compress, consolidate, or age out low-value knowledge to stay within limits.
- **Domain limits**: The user can restrict autonomous learning to specific domains or topics, or exclude sensitive domains entirely.
- **Frequency limits**: The user can configure how often autonomous learning activates (e.g., only during specific hours, only when idle for N minutes).
- **User permission**: Autonomous learning is **opt-in by default**. The user must explicitly enable it, and can disable it at any time. All autonomous learning activity is logged and reported.

These controls reflect the principle that autonomy must be bounded by transparency and user control. The system should never learn, store, or act on information without the user awareness and consent.

XII. Offline-First Design

GIA is designed to remain useful without an internet connection. This is a direct consequence of the knowledge-first architecture: because knowledge is stored locally in structured form, the system can reason, answer questions, and apply skills using only its local resources.

A. Offline Capabilities

- **Memory**: All memory types (episodic, semantic, procedural, skill) are stored locally and accessible offline.

- **Knowledge**: The knowledge graph is a local data structure, fully queryable without network access.
- **Skills**: Acquired skills are stored locally and can be retrieved and applied to new tasks.
- **Reasoning**: The Reasoning Engine operates over local knowledge and memory, requiring no external resources.
- **Previous experiences**: Episodic memory of past interactions is available for recall and analogy.

B. Online Capabilities

- **Research**: Web search and document retrieval for filling knowledge gaps.
- **Updates**: Acquiring current information (e.g., today news, current prices, latest documentation).
- **Verification**: Cross-checking local knowledge against external sources.
- **Skill expansion**: Learning new procedures or methods from external sources.

The offline-first design has practical implications. A user working in an environment without reliable internet (remote areas, air-gapped systems, privacy-sensitive settings) still has access to the system full accumulated knowledge and skills. The system degrades gracefully: when offline, it uses what it knows; when online, it can expand its knowledge. This contrasts with cloud-dependent AI systems that become non-functional without connectivity.

XIII. Privacy Model

Privacy is a first-class design principle in GIA, not an afterthought. The architecture is built around the principle that the user owns their data and controls how it is used.

- **Local-first storage**: All user data, memory, knowledge, and skills are stored locally on the user device. No data is transmitted to external servers by default.
- **Minimal external data**: When web research is necessary, only the specific query is sent externally. Retrieved information is processed locally. No user context accompanies the query.
- **No default telemetry**: The system does not collect or transmit usage analytics, performance metrics, or interaction logs by default. Any telemetry must be explicitly enabled.
- **No unnecessary cloud dependency**: The system operates fully without cloud services. Cloud connectivity is used only for web research, which is optional.
- **User-controlled memory**: The user can inspect, modify, export, and delete any stored memory, knowledge, or skill. All stored data is transparent and accessible.
- **User-controlled deletion**: The user can delete specific memories, knowledge entries, or entire knowledge domains. Deletion is immediate and irreversible.
- **User-controlled autonomous learning**: Autonomous learning is opt-in and fully configurable. The user controls what topics, how often, and under what resource constraints.

This privacy model stands in contrast to cloud-based AI services, where user data is typically transmitted to, stored on, and processed by remote servers. GIA local-first approach means that sensitive information never leaves the user device unless the user explicitly chooses to research it externally.

XIV. Computational Model

GIA is designed to operate on commodity CPU hardware, in contrast to the GPU clusters typically required for training and running large language models. This section discusses the computational tradeoffs of the architecture.

A. Potential Advantages

The primary computational advantage of GIA is not raw speed, but **amortisation**. Once knowledge and skills are locally available, repeated or related tasks may avoid repeated web retrieval and expensive neural inference. The cost of solving a problem is paid once during the first encounter; subsequent similar problems can leverage stored knowledge and skills, potentially reducing response time and computational cost.

For tasks that the system has encountered before, the computational path is: query the knowledge graph (fast, local), retrieve relevant skills (fast, local), and apply them. This contrasts with the standard LLM path of processing the full input through a large neural network for every query. However, we are careful not to claim that GIA is universally faster than LLMs; for novel tasks requiring complex language generation or deep reasoning over large contexts, a large neural model may be more efficient.

B. Resource Tradeoffs

Resource	LLM Approach	GIA Approach
CPU	Inference offloaded to GPU	Primary processing on CPU
GPU	Required (high-end for large models)	Not required
RAM	Model weights (GB-TB)	Knowledge graph + index (MB-GB)
Storage	Static (model weights)	Growing (knowledge + skills)
Network	Per-query (cloud inference)	On-demand (research only)

Table III. Resource comparison between LLM-based and GIA approaches.

The key tradeoff is between **static but large** resource requirements (LLM model weights loaded once) and **growing but bounded** resource requirements (GIA knowledge graph expanding over time). GIA storage grows with accumulated knowledge, which must be managed through compression, consolidation, and aging. The system network usage is lower for repeated tasks but potentially higher during initial knowledge acquisition. These tradeoffs are domain- and task-dependent, and empirical evaluation is needed to quantify them.

XV. Prototype Implementation

A prototype of GIA is being developed to enable empirical evaluation of the architecture. The prototype is implemented in Python, chosen for its readability, extensive library ecosystem, and suitability for research prototypes.

A. Technology Stack

- **Python**: Core implementation language for all engines and the orchestrator.
- **SQLite**: Persistent storage for episodic memory, semantic memory, and metadata. SQLite is lightweight, serverless, and suitable for local-first deployment.
- **Knowledge Graph**: Implemented as a graph structure with nodes (concepts) and typed edges (relationships), stored in SQLite with JSON-serialised attributes.
- **Local indexing**: Full-text and semantic indexing of stored knowledge for efficient retrieval, using lightweight indexing libraries.
- **Web search adapters**: Pluggable adapters for external search APIs, abstracted behind a common interface to enable testing with different search providers.
- **Verification pipeline**: A multi-stage pipeline implementing claim extraction, cross-comparison, source reputation, and confidence assignment.
- **Desktop UI**: A desktop interface for user interaction, memory inspection, knowledge graph visualisation, and autonomous learning configuration.

B. Modular Design

The prototype follows a strict modular design. Each of the fifteen components is implemented as an independent module with a well-defined interface. Components communicate through the central orchestrator, which routes data and control flow between them. This modularity enables:

- Independent testing of each component.
- Swappable implementations (e.g., different search providers, different reasoning strategies).
- Incremental development and improvement of individual components.
- Clear separation of concerns for research and experimentation.

The prototype is not intended as a production system. Its purpose is to enable empirical evaluation of the GIA hypothesis and to identify practical challenges in implementing the architecture.

XVI. Experimental Design

To evaluate the GIA hypothesis, we propose a comparative experimental design involving four system configurations, measured across ten metrics.

A. System Configurations

A. Traditional local small model: A small language model running locally, serving as a baseline. Represents the current state of local AI without persistent memory or knowledge graphs.

B. RAG system: A retrieval-augmented generation system combining a local model with a document retrieval index. Represents the current standard for knowledge-augmented local AI.

C. GIA prototype: The Genesis Intelligence Architecture as described in this paper, without a neural language model. Tests the pure knowledge-first approach.

D. GIA + lightweight language engine: GIA augmented with a small local language model for natural language understanding and generation. Tests the hybrid approach.

B. Metrics

- **Response latency**: Time from query submission to response completion, measured per task.
- **CPU usage**: Average and peak CPU utilisation during task processing.
- **RAM usage**: Average and peak memory consumption during task processing.
- **Storage growth**: Increase in storage size over time as knowledge and skills accumulate.
- **Knowledge reuse**: Proportion of tasks where previously stored knowledge or skills were successfully applied.
- **Task success**: Binary or graded success measure per task, evaluated against defined success criteria.
- **Repeated-task latency**: Latency for tasks similar to previously completed ones, measuring the benefit of accumulated experience.
- **Source verification accuracy**: Accuracy of the verification engine in correctly identifying true vs. false claims.
- **Contradiction handling**: Accuracy in detecting and appropriately resolving contradictory information.
- **Offline capability**: Proportion of tasks successfully completed without internet access.

The experimental design controls for task type, difficulty, and domain. Each system configuration is tested on an identical sequence of tasks, and measurements are compared statistically. The primary hypothesis under test is that configurations C and D will show improved repeated-task latency and knowledge reuse compared to A and B, while maintaining competitive task success rates. We do not predict universal superiority; the goal is to identify where the knowledge-first approach offers advantages and where it does not.

XVII. Example Experiment

The following experiment illustrates how GIA progressive learning would be evaluated. A sequence of related programming tasks is administered over seven days, and the system behaviour is tracked to measure whether previously learned concepts reduce research requirements and response time.

A. Task Sequence

Day 1: Create Python calculator — *Baseline: full research needed for arithmetic parsing, operator precedence, I/O handling.*

Day 2: Create GUI calculator — *Partial reuse: arithmetic logic from Day 1. New research: Tkinter layout, event handling.*

Day 3: Add history feature — *Reuses calculator + GUI structure. New: state management, history display.*

Day 7: Create scientific calculator — *Reuses calculator logic, GUI, history. New: math functions, expression evaluation.*

B. Measurements

For each task, the following are recorded: (1) number of web research queries issued, (2) response latency, (3) CPU and RAM usage, (4) number of skills reused, (5) number of new knowledge units created, and (6) task success. The expected pattern, if the hypothesis holds, is that research queries and response latency decrease for later tasks that can leverage accumulated knowledge, while task success remains stable or improves.

This experiment is designed to test the core claim of GIA: that structured experience accumulation leads to measurable improvements in efficiency for related tasks. It does not test overall intelligence or capability; it tests the *amortisation* hypothesis specifically.

XVIII. Limitations

We are explicit about the limitations of the GIA architecture, as intellectual honesty is essential for a credible research proposal.

- **Natural language understanding limitations**: GIA knowledge-first approach does not inherently possess the deep linguistic understanding of a large pretrained language model. Parsing complex, ambiguous, or idiomatic language remains challenging without a capable language engine.
- **Ambiguity**: Natural language is inherently ambiguous. Without the broad linguistic coverage of a large LLM, GIA may struggle with tasks requiring nuanced interpretation of user intent.
- **Reasoning limitations**: The Reasoning Engine operates over the knowledge graph using logical inference, analogy, and pattern matching. This is powerful for structured reasoning but limited compared to the implicit, distributed reasoning of large neural models.
- **Misinformation**: If the verification pipeline fails to detect false information, misinformation enters the knowledge graph and may be propagated. Confidence scoring and contradiction detection mitigate but cannot eliminate this risk.
- **Web-source quality**: The quality of knowledge acquired from the web depends on the quality of available sources. Even with source reputation modelling, the system may encounter domains where reliable sources are scarce.
- **Knowledge graph complexity**: As the knowledge graph grows, maintaining consistency, avoiding redundancy, and managing contradictions becomes increasingly complex. Graph traversal performance may degrade with scale.
- **Storage growth**: Unlike a fixed-size neural model, GIA storage grows with accumulated knowledge. Without effective compression, consolidation, and aging strategies, storage requirements may become impractical.
- **Autonomous learning risks**: Despite safety controls, autonomous learning carries risks: the system may research irrelevant topics, acquire incorrect information, or consume resources inappropriately.
- **Absence of large-scale pretrained linguistic knowledge**: GIA does not benefit from the vast linguistic knowledge encoded in pretrained LLM weights. Its baseline language capabilities are limited unless augmented with a language engine.
- **Difficulty achieving human-level language generation**: Generating fluent, contextually appropriate natural language text is a core strength of LLMs that GIA knowledge-first approach does not replicate.
- **Need for future experimentation**: The architecture proposed in this paper is a design and prototype. No empirical results are yet available to confirm the hypothesis. All claims about potential advantages are speculative until tested.

XIX. Security

A system that retrieves information from the web, stores it, and acts on it faces several security threats. GIA design must address these to be safe for deployment.

A. Threat Model

- **Malicious web content**: Web pages may contain intentionally misleading information designed to manipulate the system or poison its knowledge base.
- **Prompt injection from web pages**: Retrieved web content may contain instructions designed to override the system behaviour, causing it to execute unauthorised actions.
- **Poisoned knowledge**: Adversaries may publish false information specifically to influence AI systems that learn from the web. If this information passes verification, it corrupts the knowledge graph.
- **Incorrect sources**: Even without malicious intent, sources may contain errors. The system must not treat any single source as infallible.
- **Unsafe tool execution**: The Tool Engine can execute code and perform file operations. Malicious or buggy tool execution could harm the user system.
- **Autonomous actions**: The Curiosity Engine can initiate actions without direct user supervision. If compromised, autonomous learning could research sensitive topics or consume excessive resources.

B. Mitigations

- **Content sanitisation**: All retrieved web content is stripped of executable code, scripts, and embedded instructions before processing. The system processes extracted text claims, not raw HTML.
- **Claim-level processing**: Rather than feeding raw documents to a language model, GIA extracts discrete factual claims and processes them individually, reducing the surface area for prompt injection.
- **Multi-source verification**: No single source can establish a fact. Claims require corroboration from independent sources, and source reputation is tracked over time.

- **Confidence thresholds**: Knowledge below a confidence threshold is not used for critical decisions and is flagged for potential review.
- **Tool sandboxing**: Code execution and file operations are performed in a sandboxed environment with restricted permissions. Destructive operations require explicit user approval.
- **Autonomous learning bounds**: The Curiosity Engine operates within strict resource, domain, and frequency limits. All autonomous activity is logged and reviewable.
- **User override**: The user can always inspect, modify, or delete any stored knowledge, disable autonomous learning, and review the system activity log.

XX. Future Work

GIA opens several directions for future research. The following are identified as promising areas for development:

- **Better symbolic reasoning**: Developing more sophisticated reasoning engines that can handle multi-step inference, abductive reasoning, and uncertainty-aware logical operations over the knowledge graph.
- **Lightweight language understanding**: Exploring small, efficient language models that can provide natural language understanding and generation capabilities without the resource requirements of large LLMs.
- **Neural-symbolic hybrid**: Investigating deeper integration of neural and symbolic components, where a neural model handles language understanding while the symbolic system handles knowledge representation and reasoning.
- **Better knowledge compression**: Developing techniques to compress, consolidate, and abstract knowledge to manage storage growth and improve retrieval efficiency.
- **Automatic concept abstraction**: Enabling the system to automatically identify higher-level concepts from specific instances, building hierarchical knowledge structures.
- **Skill composition**: Developing mechanisms for composing multiple skills to solve complex, multi-step problems that no single skill can address alone.
- **Self-evaluation**: Implementing mechanisms for the system to evaluate its own performance, identify weaknesses, and prioritise improvement areas.
- **Knowledge aging**: Developing time-aware confidence decay models so that time-sensitive knowledge gradually loses confidence unless refreshed.
- **Source reputation**: Building more sophisticated source reputation models that track reliability across domains and over time.
- **Distributed personal knowledge**: Exploring how personal knowledge graphs could be shared, federated, or synchronised across devices while preserving privacy.
- **Hardware optimization**: Optimising knowledge graph traversal, indexing, and retrieval for specific hardware profiles, including embedded and edge devices.

XXI. Discussion

The central question that GIA raises is philosophical as well as technical: does intelligence necessarily need to be represented entirely inside neural-network weights? Current AI practice implicitly answers yes, measuring progress primarily by model size, parameter count, and training data volume. GIA explores the alternative hypothesis that useful intelligence can be distributed across multiple substrates.

Consider what makes a person effective at their work. It is not only the raw computational power of their brain but also their accumulated knowledge, their memory of past experiences, their repertoire of skills, their ability to reason by analogy, and their capacity to learn from feedback. A person who has solved similar problems before does not need to reason from first principles each time. This observation motivates the hypothesis that structured experience, not just model capacity, contributes to intelligent behaviour.

GIA proposes that useful intelligence can be distributed across:

- **Memory**: the ability to recall past events, facts, and experiences.
- **Knowledge**: structured representations of concepts and their relationships.
- **Relationships**: the connections between concepts that enable analogy and transfer.
- **Reasoning**: the ability to derive new knowledge from existing knowledge.
- **Experience**: accumulated episodic records of past problem-solving.
- **Skills**: reusable procedures and methods abstracted from successful solutions.

- **Language mechanisms**: the ability to understand and generate natural language, which may be provided by a lightweight language engine.

We are careful to distinguish this philosophical discussion from experimentally demonstrated results. The hypothesis that distributed intelligence across these substrates can yield progressively more capable AI is plausible and motivated by existing research in memory-augmented networks [7], [8], knowledge graphs [11], continual learning [5], [6], and neuro-symbolic AI [9], [10]. However, it has not been empirically validated in the form proposed by GIA. The experimental design in Section XVI is intended to provide such validation.

If the hypothesis is partially confirmed, the implications could be significant. It would suggest that the path to more capable AI is not solely through larger models, but also through richer knowledge structures, more sophisticated memory, and better integration of experience. This does not diminish the importance of neural models; rather, it suggests that neural models and knowledge-first architectures may be complementary, each addressing what the other cannot.

XXII. Conclusion

This paper has proposed the Genesis Intelligence Architecture (GIA), a lightweight, knowledge-first, experience-driven approach to artificial intelligence. GIA shifts the primary mechanism of growth from increasing model weights to persistent memory, structured knowledge graphs, experience accumulation, verification, reasoning, skill acquisition, and controlled autonomous learning. The architecture is designed to operate on commodity CPU hardware and to remain useful both offline and online.

The central research question posed by GIA is not:

How do we make the model larger?

but rather:

How can the system become more capable through accumulated experience?

GIA does not claim to replace large language models, to have achieved human-level intelligence, or to be superior to existing systems. It is a proposed research direction, presented as an experimental architecture and a research proposal. The architecture is designed to be testable, and the experimental design in Section XVI outlines how the core hypothesis can be empirically evaluated.

The contribution of this paper is to articulate a scientifically credible foundation for a knowledge-first, experience-driven approach to AI, to clearly distinguish existing technologies from the proposed contribution, and to outline a path toward empirical validation. Whether this approach yields meaningful advantages over existing architectures is a question that only experimentation can answer. We present GIA in the spirit of open inquiry, as a hypothesis to be tested rather than a conclusion to be accepted.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. 31st Int. Conf. Neural Information Processing Systems (NIPS), Long Beach, CA, USA, 2017, pp. 6000-6010.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, et al., "Language models are few-shot learners," in Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS), Vancouver, Canada, 2020, pp. 1877-1901.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS), Vancouver, Canada, 2020, pp. 9459-9474.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), Kigali, Rwanda, 2023.

[5] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 44, no. 7, pp. 3366-3385, Jul. 2022.

[6] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 46, no. 8, pp. 5362-5383, Aug. 2024.

[7] J. Weston, S. Chopra, and A. Bordes, "Memory networks," in Proc. Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015.

[8] A. Graves, G. Wayne, and I. Danihelka, "Neural Turing machines," arXiv preprint arXiv:1410.5401, 2014.

[9] A. d'Avila Garcez, T. R. Besold, et al., "Neural-symbolic learning and reasoning: A survey and interpretation," in Neuro-Symbolic Artificial Intelligence: The State of the Art, IOS Press, 2022, pp. 327-352.

[10] I. Nieves-Rosendo, J. Del Ser, A. Villarrubia, et al., "Neuro-symbolic artificial intelligence: A survey," Neural Computing and Applications, vol. 36, pp. 1-26, Jun. 2024.

[11] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, et al., "Knowledge graphs," ACM Computing Surveys, vol. 54, no. 4, pp. 1-37, Jul. 2021.